SERVICE DESK API

Relatório de Testes das Rotas

Execução via Swagger UI | Ambiente Docker

23/04/2026

1. Resumo Executivo

Rotas Testadas	Aprovadas	Falhas	Taxa de Sucesso
6	6	0	100%

Todos os endpoints da API Service Desk foram testados com sucesso utilizando a interface Swagger UI, disponível automaticamente pelo FastAPI em modo debug. Os testes foram executados em ambiente Docker local, com banco de dados MySQL em container dedicado.

2. Ambiente de Teste

2.1 Tecnologias Utilizadas

Interface de Teste	`Swagger UI (FastAPI /docs)`

URL Base	`http://localhost:8000`

Infraestrutura	`Docker Desktop + docker compose`

Banco de Dados	`MySQL 8.0 (container Docker)`

Framework API	`FastAPI 0.135.2 + Python 3.12`

Sistema Operacional	`Windows 11 (execução local)`

2.2 Como o Ambiente Foi Configurado

O ambiente foi montado do zero seguindo os passos abaixo:

1 **Instalação do Docker Desktop**
Download e instalação do Docker Desktop para Windows. Foram necessários ajustes de permissão na pasta C:\ProgramData\DockerDesktop via PowerShell como administrador.

2 **Criação dos arquivos Docker**
Criação do Dockerfile e docker-compose.yml diretamente pelo PowerShell, configurando o serviço de banco MySQL e a API FastAPI com variáveis de ambiente.

3 **Ajuste nas migrations**
O projeto continha um conflito de índice duplicado nas migrations do Alembic. A solução foi substituir o comando alembic upgrade head pela criação direta das tabelas via SQLAlchemy (Base.metadata.create_all).

4 **Subida do ambiente**
Execução do comando docker compose up --build, que baixou as imagens, criou as tabelas no banco e iniciou o servidor uvicorn na porta 8000.

5 **Acesso ao Swagger UI**
Abertura do navegador em http://localhost:8000/docs para acesso a interface visual de testes.

2.3 Comandos Executados

Sequência de comandos utilizados no PowerShell para subir o ambiente:

```
# Limpar volumes anteriores (banco zerado)
docker compose down -v
```

```
# Construir imagens e subir os containers
docker compose up --build

# Saida esperada ao iniciar com sucesso:
# service_desk_api | INFO: Application startup complete.
# service_desk_api | INFO: Uvicorn running on http://0.0.0.0:8000
```

3. Execução dos Testes

Os testes foram realizados manualmente pela interface Swagger UI, clicando em cada rota, ativando 'Try it out', preenchendo os dados e executando. Os resultados foram registrados conforme retorno da API.

3.1 GET /health — Health Check

Verifica se a aplicação está no ar e se o banco de dados está conectado.

Dados enviados:

Nenhum parâmetro necessário.

Resposta recebida (HTTP 200):

```
HTTP 200 OK
{
  "status": "ok",
  "db": "connected"
}
```

Resultado: Banco de dados conectado e API funcionando corretamente.

3.2 POST /tickets — Criar Ticket

Criação de um novo ticket de suporte com título, descrição, prioridade e categoria.

Dados enviados (Request Body):

```
{
  "title": "Problema no VPN",
  "description": "Usuario nao consegue conectar ao VPN da empresa",
  "priority": "high",
  "category": "rede"
}
```

Resposta recebida (HTTP 201 Created):

```
HTTP 201 Created
{
  "id": "bb44df72-6b7d-4d9d-955e-b25a879c1adc",
  "title": "Problema no VPN",
  "description": "Usuario nao consegue conectar ao VPN da empresa",
  "status": "pending",
  "priority": "high",
```

```
  "user_id": null,
  "client_id": null,
  "assigned_to": null,
  "updated_by": null,
  "category": "rede",
  "created_at": "2026-04-23T22:17:50",
  "updated_at": "2026-04-23T22:17:50",
  "closed_at": null
}
```

Resultado: Ticket criado com UUID gerado automaticamente e status inicial 'pending'.

3.3 GET /tickets — Listar Tickets

Listagem paginada de todos os tickets cadastrados no sistema.

Parametros utilizados:

```
skip=0  |  limit=50  (valores padrao)
```

Resposta recebida (HTTP 200 OK):

```
HTTP 200 OK
{
  "total": 2,
  "items": [
    {
      "id": "bb44df72-6b7d-4d9d-955e-b25a879c1adc",
      "title": "Problema no VPN",
      "status": "pending",
      "priority": "high",
      "category": "rede",
      "created_at": "2026-04-23T22:17:50",
      ...
    },
    { ... segundo ticket ... }
  ]
}
```

Resultado: Retornou os 2 tickets criados durante o teste com paginacao funcionando.

3.4 GET /tickets/{ticket_id} — Buscar Ticket por ID

Busca de um ticket especifico pelo seu UUID.

Parâmetro utilizado:

```
ticket_id: bb44df72-6b7d-4d9d-955e-b25a879c1adc
```

Resposta recebida (HTTP 200 OK):

```
HTTP 200 OK
{
  "id": "bb44df72-6b7d-4d9d-955e-b25a879c1adc",
  "title": "Problema no VPN",
  "description": "Usuario nao consegue conectar ao VPN da empresa",
  "status": "pending",
  "priority": "high",
  "category": "rede",
  "created_at": "2026-04-23T22:17:50",
  "updated_at": "2026-04-23T22:17:50",
  "closed_at": null
}
```

Resultado: Ticket encontrado e retornado corretamente pelo UUID.

3.5 PATCH /tickets/{ticket_id} — Atualizar Ticket

Atualização parcial de um ticket existente. Apenas os campos enviados sao alterados (partial update).

Parâmetro de rota:

```
ticket_id: bb44df72-6b7d-4d9d-955e-b25a879c1adc
```

Dados enviados (Request Body):

```
{
  "title": "string",
  "description": "stringstri",
  "status": "pending",
  "priority": "low",
  "assigned_to": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "updated_by": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "category": "string"
}
```

Resposta recebida (HTTP 200 OK):

```
HTTP 200 OK
{
  "id": "bb44df72-6b7d-4d9d-955e-b25a879c1adc",
  "title": "string",
  "description": "stringstri",
  "status": "pending",
  "priority": "low",
  "assigned_to": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "updated_by": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "category": "string",
  "created_at": "2026-04-23T22:17:50",
  "updated_at": "2026-04-23T22:20:30",
  "closed_at": null
}
```

Resultado: Campos atualizados com sucesso. O campo updated_at foi atualizado automaticamente de 22:17:50 para 22:20:30.

3.6 GET /knowledge-base — Base de Conhecimento

Lista tickets encerrados (status 'done') disponibilizados como base de conhecimento para consulta futura.

Parametros utilizados:

```
skip=0  |  limit=50  (valores padrao)
```

Resposta recebida (HTTP 200 OK):

```
HTTP 200 OK
{
  "total": 0,
  "items": []
}
```

Resultado: Retornou lista vazia, pois nenhum ticket foi marcado como 'done' durante os testes. Comportamento esperado e correto.

4. Tabela Resumo dos Resultados

Metodo	Rota	Dados Enviados	HTTP	Status
GET	/health	Nenhum parâmetro	200 OK	✓ OK
POST	/tickets	title, description, priority, category	201	✓ OK
GET	/tickets	skip=0, limit=50	200 OK	✓ OK
GET	/tickets/{id}	ticket_id (UUID no path)	200 OK	✓ OK
PATCH	/tickets/{id}	title, description, status, priority, ...	200 OK	✓ OK
GET	/knowledge-base	skip=0, limit=50	200 OK	✓ OK

5. Observações e Conclusões

5.1 Comportamentos Validados

- UUID gerado automaticamente ao criar ticket (campo id)
- Campo created_at preenchido no momento da criacao
- Campo updated_at atualizado automaticamente ao fazer PATCH
- Status inicial padrao 'pending' aplicado corretamente
- Paginacao funcionando com os parametros skip e limit
- Retorno de lista vazia na knowledge-base quando nao ha tickets encerrados
- Codigo HTTP correto em cada operacao: 201 para criacao, 200 para leitura/atualizacao

5.2 Pontos de Atenção

- O projeto tinha um bug nas migrations do Alembic (indice duplicado ix_ticket_messages_author_id). Foi necessario contornar usando Base.metadata.create_all diretamente.
- A knowledge-base so exibira dados quando um ticket for atualizado para status 'done'.
- Os campos user_id, client_id, assigned_to e updated_by sao opcionais e retornam null quando nao informados.

5.3 Conclusão

A API Service Desk está funcionando corretamente em ambiente Docker local. Todas as 6 rotas testadas retornaram os códigos HTTP esperados e os dados no formato correto. O ambiente está pronto para uso e testes adicionais.

✓ **Todos os testes concluídos com sucesso — 6/6 rotas aprovadas**